Neural Mutual Information Estimation with Vector Copulas

Michael U. Gutmann

michael.gutmann@ed.ac.uk

School of Informatics, University of Edinburgh

18 March 2026

Acknowledgements

The presentation is based on:

Neural Mutual Information Estimation with Vector Copulas
Yanzhi Chen, Zijing Ou, Adrian Weller, Michael U. Gutmann
NeurIPS 2025
`https://openreview.net/pdf?id=FYbe7r0mxu`

Contents

Motivation

Background

New MI estimation framework

Simulation results

Program

Motivation

Background

New MI estimation framework

Simulation results

Mutual information (MI)

- The MI between two vector-valued random variables $\mathbf{x}$ and $\mathbf{y}$ is the Kullback-Leibler (KL) divergence between their joint $p(\mathbf{x}, \mathbf{y})$ and the product of their marginals, $p(\mathbf{x})p(\mathbf{y})$:

$$I(\mathbf{x}; \mathbf{y}) = \mathrm{KL}\left[p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})\right] = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \quad (1)$$

Mutual information (MI)

- The MI between two vector-valued random variables $\mathbf{x}$ and $\mathbf{y}$ is the Kullback-Leibler (KL) divergence between their joint $p(\mathbf{x}, \mathbf{y})$ and the product of their marginals, $p(\mathbf{x})p(\mathbf{y})$:

$$I(\mathbf{x}; \mathbf{y}) = \mathrm{KL}\left[p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})\right] = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \quad (1)$$

- It measures the dependence between $\mathbf{x}$ and $\mathbf{y}$.

Mutual information (MI)

- The MI between two vector-valued random variables $\mathbf{x}$ and $\mathbf{y}$ is the Kullback-Leibler (KL) divergence between their joint $p(\mathbf{x}, \mathbf{y})$ and the product of their marginals, $p(\mathbf{x})p(\mathbf{y})$:

$$I(\mathbf{x}; \mathbf{y}) = \mathrm{KL}\left[p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})\right] = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \quad (1)$$

- It measures the dependence between $\mathbf{x}$ and $\mathbf{y}$.
- The MI is zero if $\mathbf{x}$ and $\mathbf{y}$ are statistically independent.

Mutual information (MI)

- The MI between two vector-valued random variables $\mathbf{x}$ and $\mathbf{y}$ is the Kullback-Leibler (KL) divergence between their joint $p(\mathbf{x}, \mathbf{y})$ and the product of their marginals, $p(\mathbf{x})p(\mathbf{y})$:

$$I(\mathbf{x}; \mathbf{y}) = \mathrm{KL}\left[p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})\right] = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \quad (1)$$

- It measures the dependence between $\mathbf{x}$ and $\mathbf{y}$.
- The MI is zero if $\mathbf{x}$ and $\mathbf{y}$ are statistically independent.
- Quantifies how well one can predict $\mathbf{y}$ from $\mathbf{x}$ and vice versa.

Mutual information (MI)

- The MI between two vector-valued random variables $\mathbf{x}$ and $\mathbf{y}$ is the Kullback-Leibler (KL) divergence between their joint $p(\mathbf{x}, \mathbf{y})$ and the product of their marginals, $p(\mathbf{x})p(\mathbf{y})$:

$$I(\mathbf{x}; \mathbf{y}) = \mathrm{KL}\left[p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})\right] = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \quad (1)$$

- It measures the dependence between $\mathbf{x}$ and $\mathbf{y}$.
- The MI is zero if $\mathbf{x}$ and $\mathbf{y}$ are statistically independent.
- Quantifies how well one can predict $\mathbf{y}$ from $\mathbf{x}$ and vice versa.
- Problem considered: Estimate $I(\mathbf{x}; \mathbf{y})$ from a dataset $\mathcal{D} = \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^{n}$, with $\mathbf{x}^{(i)}, \mathbf{y}^{(i)} \sim p(\mathbf{x}, \mathbf{y})$.

Mutual information (MI)

- The MI between two vector-valued random variables $\mathbf{x}$ and $\mathbf{y}$ is the Kullback-Leibler (KL) divergence between their joint $p(\mathbf{x}, \mathbf{y})$ and the product of their marginals, $p(\mathbf{x})p(\mathbf{y})$:

$$I(\mathbf{x}; \mathbf{y}) = \mathrm{KL}\left[p(\mathbf{x}, \mathbf{y}) \| p(\mathbf{x})p(\mathbf{y})\right] = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \quad (1)$$

- It measures the dependence between $\mathbf{x}$ and $\mathbf{y}$.
- The MI is zero if $\mathbf{x}$ and $\mathbf{y}$ are statistically independent.
- Quantifies how well one can predict $\mathbf{y}$ from $\mathbf{x}$ and vice versa.
- Problem considered: Estimate $I(\mathbf{x}; \mathbf{y})$ from a dataset $\mathcal{D} = \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^{n}$, with $\mathbf{x}^{(i)}, \mathbf{y}^{(i)} \sim p(\mathbf{x}, \mathbf{y})$.
- Consider the case of continuous random variables.

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.
- Feature learning (Tishby et al, 1999): find minimal representations that are most predictive about the target.

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.
- Feature learning (Tishby et al, 1999): find minimal representations that are most predictive about the target.
- Algorithmic fairness (Chen et al, 2022): find representations that do not leak information about protected attributes

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.
- Feature learning (Tishby et al, 1999): find minimal representations that are most predictive about the target.
- Algorithmic fairness (Chen et al, 2022): find representations that do not leak information about protected attributes
- Applications in biology (Gowri et al. 2024)

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.
- Feature learning (Tishby et al, 1999): find minimal representations that are most predictive about the target.
- Algorithmic fairness (Chen et al, 2022): find representations that do not leak information about protected attributes
- Applications in biology (Gowri et al. 2024)
  - Quantify protein-protein interaction

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.
- Feature learning (Tishby et al, 1999): find minimal representations that are most predictive about the target.
- Algorithmic fairness (Chen et al, 2022): find representations that do not leak information about protected attributes
- Applications in biology (Gowri et al. 2024)
  - Quantify protein-protein interaction
  - Quantify how well stem cell differentiation can be predicted from gene activity patterns

Why is MI estimation important?

- MI is an abstract quantity with wide applicability.
- Experimental design (Lindley, 1956): MI to quantify how informative an experiment is.
- Feature learning (Tishby et al, 1999): find minimal representations that are most predictive about the target.
- Algorithmic fairness (Chen et al, 2022): find representations that do not leak information about protected attributes
- Applications in biology (Gowri et al. 2024)
  - Quantify protein-protein interaction
  - Quantify how well stem cell differentiation can be predicted from gene activity patterns
- ...

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Issue:

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Issue:

- Distribution-free density estimation scales poorly with dimensionality (curse of dimensionality).

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Issue:

- Distribution-free density estimation scales poorly with dimensionality (curse of dimensionality).

2. Ratio estimation: Estimate the log ratio (point-wise MI)

$$\log \left[\frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right]$$

typically done via MI lower-bound maximisation.

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Issue:

- Distribution-free density estimation scales poorly with dimensionality (curse of dimensionality).

2. Ratio estimation: Estimate the log ratio (point-wise MI)

$$\log \left[\frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right]$$

typically done via MI lower-bound maximisation.

Issue:

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Issue:

- Distribution-free density estimation scales poorly with dimensionality (curse of dimensionality).

2. Ratio estimation: Estimate the log ratio (point-wise MI)

$$\log \left[\frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right]$$

typically done via MI lower-bound maximisation.

Issue:

- Sample-size required for distribution-free estimate of MI lower bound grows exponentially with the value of the bound (McAllester and Stratos, 2020)

Previous work on estimating the MI

There are broadly two existing approaches to estimating the MI

1. Density estimation: Estimate the joint $p(\mathbf{x}, \mathbf{y})$ and the two marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ separately.

Issue:

- Distribution-free density estimation scales poorly with dimensionality (curse of dimensionality).

2. Ratio estimation: Estimate the log ratio (point-wise MI)

$$\log \left[\frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right]$$

typically done via MI lower-bound maximisation.

Issue:

- Sample-size required for distribution-free estimate of MI lower bound grows exponentially with the value of the bound (McAllester and Stratos, 2020)
- Ratio estimation becomes harder when the MI is high (density-chasm problem, Rhodes et al, 2020)

Density-chasm problem

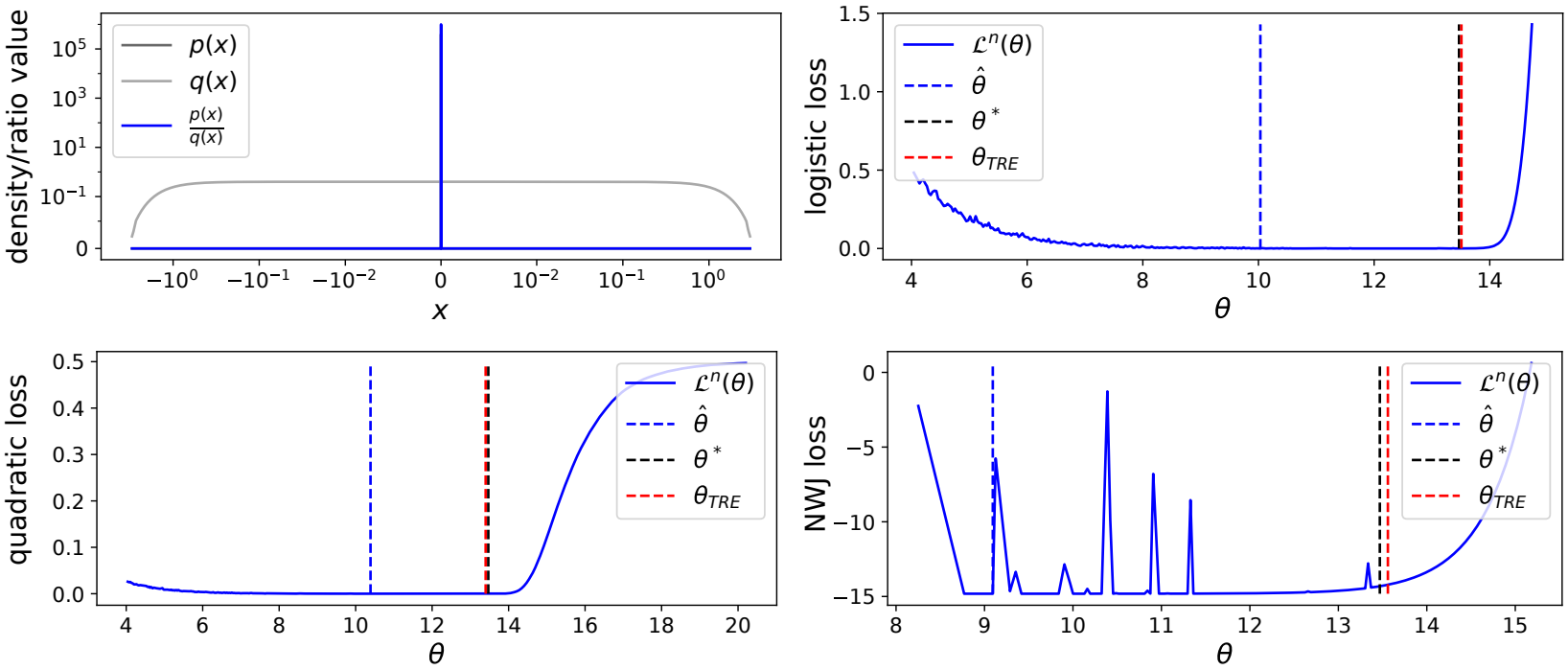
- Logistic loss and other ratio estimation methods struggle if the two distributions are very different ("density chasm")

Density-chasm problem

- Logistic loss and other ratio estimation methods struggle if the two distributions are very different ("density chasm")
- Consider ratio between two zero-mean Gaussians. 10'000 samples from each distribution. Ratio parameterised by $\theta \in \mathbb{R}$.

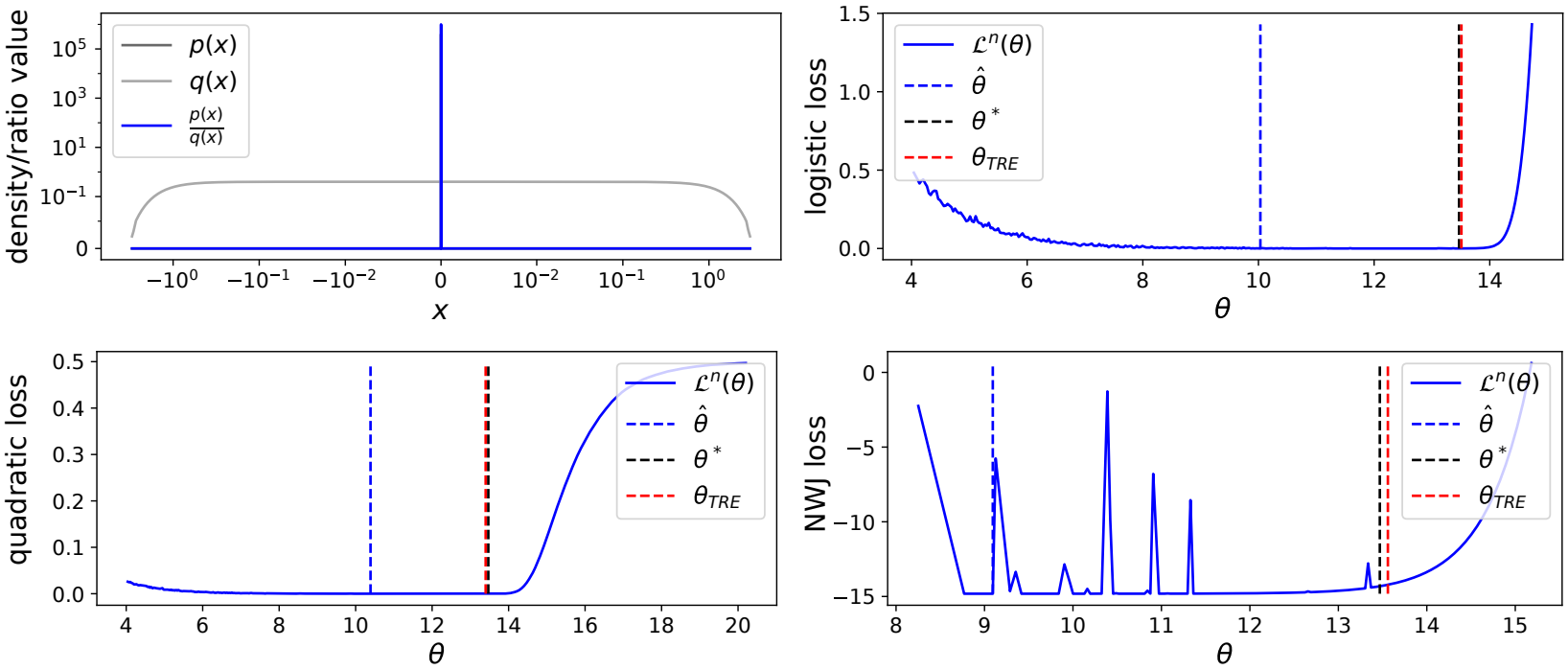
Density-chasm problem

- Logistic loss and other ratio estimation methods struggle if the two distributions are very different ("density chasm")
- Consider ratio between two zero-mean Gaussians. 10'000 samples from each distribution. Ratio parameterised by $\theta \in \mathbb{R}$.



Density-chasm problem

- Logistic loss and other ratio estimation methods struggle if the two distributions are very different ("density chasm")
- Consider ratio between two zero-mean Gaussians. 10'000 samples from each distribution. Ratio parameterised by $\theta \in \mathbb{R}$.
- Solution in red bridges the "gap" using telescopic ratio estimation (TRE) (Rhodes et al, 2020)



In need of a compromise

- Distribution-free MI estimation is only practical in low dimensions.

In need of a compromise

- Distribution-free MI estimation is only practical in low dimensions.
- Restricting the model class for $p(\mathbf{x}, \mathbf{y})$ helps, but assumptions (e.g Gaussianity) may be too restrictive.

In need of a compromise

- Distribution-free MI estimation is only practical in low dimensions.
- Restricting the model class for $p(\mathbf{x}, \mathbf{y})$ helps, but assumptions (e.g Gaussianity) may be too restrictive.
- We are here proposing a compromise: impose lightweight assumptions on the interactions between $\mathbf{x}$ and $\mathbf{y}$, but not on the entire distribution $p(\mathbf{x}, \mathbf{y})$.

In need of a compromise

- Distribution-free MI estimation is only practical in low dimensions.
- Restricting the model class for $p(\mathbf{x}, \mathbf{y})$ helps, but assumptions (e.g Gaussianity) may be too restrictive.
- We are here proposing a compromise: impose lightweight assumptions on the interactions between $\mathbf{x}$ and $\mathbf{y}$, but not on the entire distribution $p(\mathbf{x}, \mathbf{y})$.
- We model the marginals $p(\mathbf{x})$ and $p(\mathbf{y})$ flexibly but the interactions between $\mathbf{x}$ and $\mathbf{y}$ in a more controlled way, using the theory of vector copulas (Fan and Henry, J of Econometrics, 2023).

Program

Motivation

Background

New MI estimation framework

Simulation results

Background on copulas

- Sklar's theorem (Sklar, 1959): for any multivariate cdf $F(x_1, \ldots, x_d)$ with univariate cdfs $F_1, \ldots, F_d$, we have

$$F(x_1, \ldots, x_d) = C(F_1(x_1), \ldots, F_d(x_d)) \tag{2}$$

  where C is the so-called copula function, a cdf on $[0, 1]^d$.

Background on copulas

- Sklar's theorem (Sklar, 1959): for any multivariate cdf $F(x_1, \ldots, x_d)$ with univariate cdfs $F_1, \ldots, F_d$, we have

$$F(x_1, \ldots, x_d) = C(F_1(x_1), \ldots, F_d(x_d)) \tag{2}$$

  where C is the so-called copula function, a cdf on $[0, 1]^d$.
- For continuous random variables, we have

$$p(x_1, \ldots, x_d) = c(F_1(x_1), \ldots, F_d(x_d)) \prod_{i=1}^{d} p_i(x_i) \tag{3}$$

  where p and c are the densities associated with F and C.

Background on copulas

$$p(x_1, \ldots, x_d) = c(F_1(x_1), \ldots, F_d(x_d)) \prod_{i=1}^{d} p_i(x_i)$$

- Variables x_i are separately transformed to uniforms $u_i = F_i(x_i)$. This is a kind of "standardisation"; transforms x_i into ranks.

Background on copulas

$$p(x_1, \ldots, x_d) = c(F_1(x_1), \ldots, F_d(x_d)) \prod_{i=1}^{d} p_i(x_i)$$

- Variables x_i are separately transformed to uniforms $u_i = F_i(x_i)$. This is a kind of "standardisation"; transforms x_i into ranks.
- The copula models the joint distribution of the u_i.

Background on copulas

$$p(x_1, \ldots, x_d) = c(F_1(x_1), \ldots, F_d(x_d)) \prod_{i=1}^{d} p_i(x_i)$$

- Variables x_i are separately transformed to uniforms $u_i = F_i(x_i)$. This is a kind of "standardisation"; transforms x_i into ranks.
- The copula models the joint distribution of the u_i.
- By the change of variable formula, using $F_i'(x_i) = p_i(x_i)$, their joint density is

$$p_{\mathbf{u}}(u_1, \ldots, u_d) = c(u_1, \ldots, u_d) \tag{4}$$

Captures dependencies between the u_i, and thereby the x_i.

Background on copulas

$$p(x_1, \ldots, x_d) = c(F_1(x_1), \ldots, F_d(x_d)) \prod_{i=1}^{d} p_i(x_i)$$

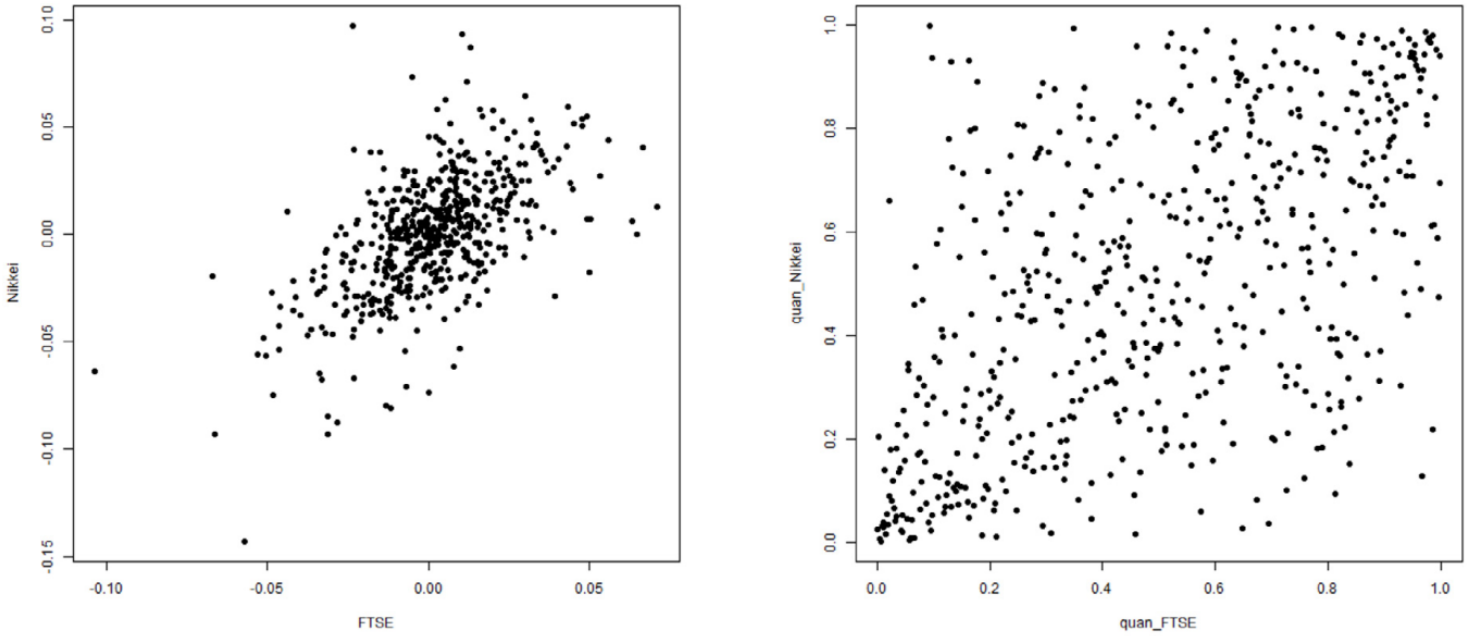
- Variables x_i are separately transformed to uniforms $u_i = F_i(x_i)$. This is a kind of "standardisation"; transforms x_i into ranks.
- The copula models the joint distribution of the u_i.
- By the change of variable formula, using $F_i'(x_i) = p_i(x_i)$, their joint density is

$$p_{\mathbf{u}}(u_1, \ldots, u_d) = c(u_1, \ldots, u_d) \tag{4}$$

  Captures dependencies between the u_i, and thereby the x_i.
- Main point: Copula decomposition isolates dependence properties from univariate marginal properties.

Background on copulas

Example (Fan and Henry, J of Econometrics, 2023)



- Left: data in original domain (the x_i). Marginals properties overshadow finer-grained dependencies.
- Right: data in transformed domain (the u_i). Dependencies for large and small u_i (tail dependencies) become visible.

Background on vector copulas

- Vector copula generalizes the traditional copula to allow for multivarate marginals.

Background on vector copulas

- Vector copula generalizes the traditional copula to allow for multivarate marginals.
- Vector Sklar theorem (Fan and Henry, 2023, simplified): Let $\mathbf{x}_i, i = 1 \ldots, k$, be continuous random vectors of dimension d_i, then there exist invertible mappings G_i and a density (vector copula) c such that

$$G_i(\mathbf{x}_i) \sim \mathcal{U}[0,1]^{d_i} \tag{5}$$

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_k) = c(G_1(\mathbf{x}_1), \ldots, G_k(\mathbf{x}_k)) \prod_{i=1}^{k} p(\mathbf{x}_i) \tag{6}$$

Background on vector copulas

- Vector copula generalizes the traditional copula to allow for multivarate marginals.
- Vector Sklar theorem (Fan and Henry, 2023, simplified): Let $\mathbf{x}_i, i = 1 \ldots, k$, be continuous random vectors of dimension d_i, then there exist invertible mappings G_i and a density (vector copula) c such that

$$G_i(\mathbf{x}_i) \sim \mathcal{U}[0,1]^{d_i} \tag{5}$$

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_k) = c(G_1(\mathbf{x}_1), \ldots, G_k(\mathbf{x}_k)) \prod_{i=1}^{k} p(\mathbf{x}_i) \tag{6}$$

- $G_i(\mathbf{x}_i)$ generalises the mappings $F(x_i)$ in the classical copulas, removing structure from $\mathbf{x}_i$ so that the vector copula only represents their dependencies, and not marginal properties.

Background on vector copulas

- Note: $\mathbf{x}_i \sim p(\mathbf{x}_i)$ and $G_i(\mathbf{x}_i) \sim \mathcal{U}[0,1]^{d_i}$ implies

$$|\det J_{G_i}(\mathbf{x}_i)| = p(\mathbf{x}_i) \tag{7}$$

Background on vector copulas

- Note: $\mathbf{x}_i \sim p(\mathbf{x}_i)$ and $G_i(\mathbf{x}_i) \sim \mathcal{U}[0,1]^{d_i}$ implies

$$|\det J_{G_i}(\mathbf{x}_i)| = p(\mathbf{x}_i) \tag{7}$$

- While the $\mathbf{u}_i = G_i(\mathbf{x}_i)$ are each marginally uniformly distributed, their joint distribution is (with $\mathbf{x}_i = G_i^{-1}(\mathbf{u}_i)$)

$$\begin{aligned}
p_{\mathbf{u}}(\mathbf{u}_1, \ldots, \mathbf{u}_k) &= p(\mathbf{x}_1, \ldots, \mathbf{x}_k) \prod_{i=1}^{k} |\det J_{G_i}(\mathbf{x}_i)|^{-1} && (8)\\
&= p(\mathbf{x}_1, \ldots, \mathbf{x}_k) \prod_{i=1}^{k} \frac{1}{p(\mathbf{x}_i)} && (9)\\
&= c(G_1(\mathbf{x}_1), \ldots, G_k(\mathbf{x}_k)) \prod_{i=1}^{k} p(\mathbf{x}_i) \prod_{i=1}^{k} \frac{1}{p(\mathbf{x}_i)} && (10)\\
&= c(\mathbf{u}_1, \ldots, \mathbf{u}_k) && (11)
\end{aligned}$$

Background on vector copulas

- Note: $\mathbf{x}_i \sim p(\mathbf{x}_i)$ and $G_i(\mathbf{x}_i) \sim \mathcal{U}[0,1]^{d_i}$ implies

$$|\det J_{G_i}(\mathbf{x}_i)| = p(\mathbf{x}_i) \tag{7}$$

- While the $\mathbf{u}_i = G_i(\mathbf{x}_i)$ are each marginally uniformly distributed, their joint distribution is (with $\mathbf{x}_i = G_i^{-1}(\mathbf{u}_i)$)

$$\begin{aligned}
p_{\mathbf{u}}(\mathbf{u}_1, \ldots, \mathbf{u}_k) &= p(\mathbf{x}_1, \ldots, \mathbf{x}_k) \prod_{i=1}^{k} |\det J_{G_i}(\mathbf{x}_i)|^{-1} && (8)\\
&= p(\mathbf{x}_1, \ldots, \mathbf{x}_k) \prod_{i=1}^{k} \frac{1}{p(\mathbf{x}_i)} && (9)\\
&= c(G_1(\mathbf{x}_1), \ldots, G_k(\mathbf{x}_k)) \prod_{i=1}^{k} p(\mathbf{x}_i) \prod_{i=1}^{k} \frac{1}{p(\mathbf{x}_i)} && (10)\\
&= c(\mathbf{u}_1, \ldots, \mathbf{u}_k) && (11)
\end{aligned}$$

- Vector copula c equals the joint distribution of the "standardised" random variables $\mathbf{u}_i$.

Program

Motivation

Background

New MI estimation framework

Simulation results

Change of perspective: it's a density, not a density ratio

- Recall the definition of the MI:

$$I(\mathbf{x};\mathbf{y}) = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x},\mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \tag{12}$$

Change of perspective: it's a density, not a density ratio

- Recall the definition of the MI:

$$I(\mathbf{x};\mathbf{y}) = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x},\mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \tag{12}$$

- With transformations G_x and G_y such that

$$\mathbf{u}_x = G_x(\mathbf{x}) \sim \mathcal{U}[0,1]^{d_x} \qquad \mathbf{u}_y = G_y(\mathbf{y}) \sim \mathcal{U}[0,1]^{d_y} \tag{13}$$

it follows from (9) on the previous slide that

$$p_{\mathbf{u}}(\mathbf{u}_x,\mathbf{u}_y) = \frac{p(\mathbf{x},\mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \tag{14}$$

Change of perspective: it's a density, not a density ratio

- Recall the definition of the MI:

$$I(\mathbf{x};\mathbf{y}) = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\log \frac{p(\mathbf{x},\mathbf{y})}{p(\mathbf{x})p(\mathbf{y})}\right] \tag{12}$$

- With transformations G_x and G_y such that

$$\mathbf{u}_x = G_x(\mathbf{x}) \sim \mathcal{U}[0,1]^{d_x} \qquad \mathbf{u}_y = G_y(\mathbf{y}) \sim \mathcal{U}[0,1]^{d_y} \tag{13}$$

  it follows from (9) on the previous slide that

$$p_{\mathbf{u}}(\mathbf{u}_x,\mathbf{u}_y) = \frac{p(\mathbf{x},\mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \tag{14}$$

- The density ratio (pointwise MI) equals the joint density of $\mathbf{u}_x, \mathbf{u}_y$, in what follows denoted by $c(\mathbf{u}_x, \mathbf{u}_y)$.

MI as negative vector copula entropy

- With the law of the unconscious statistician, the MI thus becomes

$$I(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{x},\mathbf{y}} \left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right] \tag{15}$$

$$= \mathbb{E}_{\mathbf{u}_x,\mathbf{u}_y} \left[\log c(\mathbf{u}_x, \mathbf{u}_y) \right] \tag{16}$$

with $\mathbf{u}_x = G_x(\mathbf{x})$ and $\mathbf{u}_y = G_y(\mathbf{y})$.

MI as negative vector copula entropy

- With the law of the unconscious statistician, the MI thus becomes

$$I(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{x},\mathbf{y}} \left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right] \tag{15}$$

$$= \mathbb{E}_{\mathbf{u}_x,\mathbf{u}_y} \left[\log c(\mathbf{u}_x, \mathbf{u}_y) \right] \tag{16}$$

  with $\mathbf{u}_x = G_x(\mathbf{x})$ and $\mathbf{u}_y = G_y(\mathbf{y})$.
- This is the negative entropy H of c.

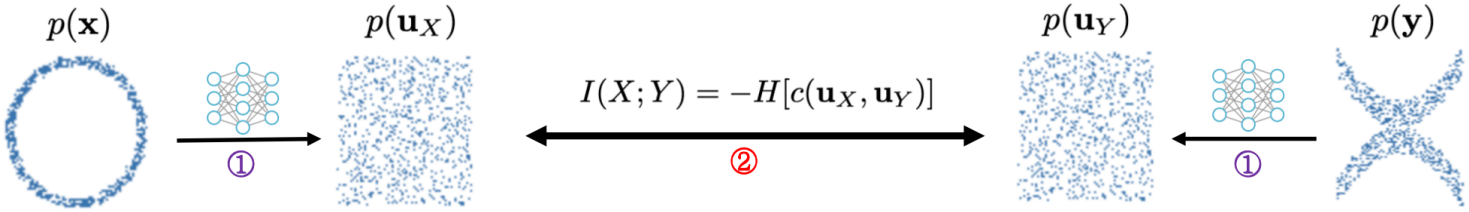
MI as negative vector copula entropy

- With the law of the unconscious statistician, the MI thus becomes

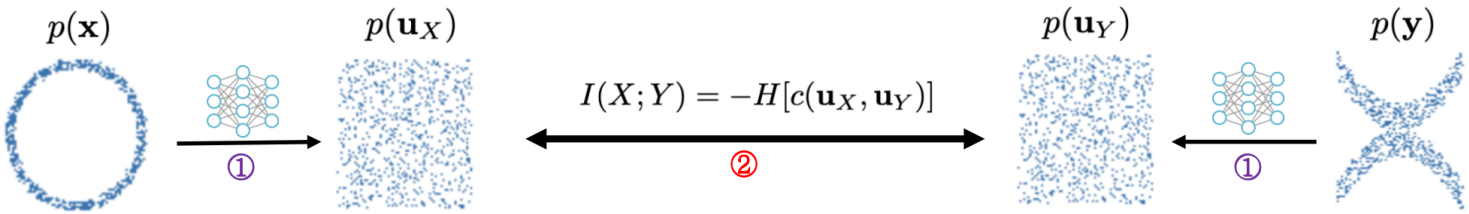
$$I(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{x},\mathbf{y}} \left[\log \frac{p(\mathbf{x}, \mathbf{y})}{p(\mathbf{x})p(\mathbf{y})} \right] \tag{15}$$

$$= \mathbb{E}_{\mathbf{u}_x,\mathbf{u}_y} \left[\log c(\mathbf{u}_x, \mathbf{u}_y) \right] \tag{16}$$

  with $\mathbf{u}_x = G_x(\mathbf{x})$ and $\mathbf{u}_y = G_y(\mathbf{y})$.
- This is the negative entropy H of c.
- Suggests a divide-and-conquer approach:

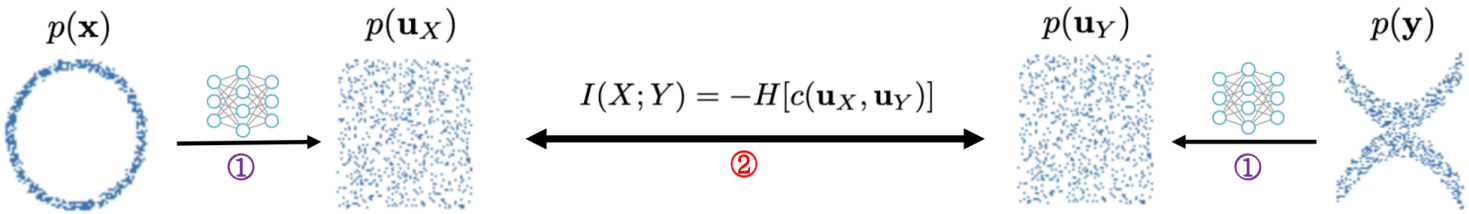


Divide and conquer approach



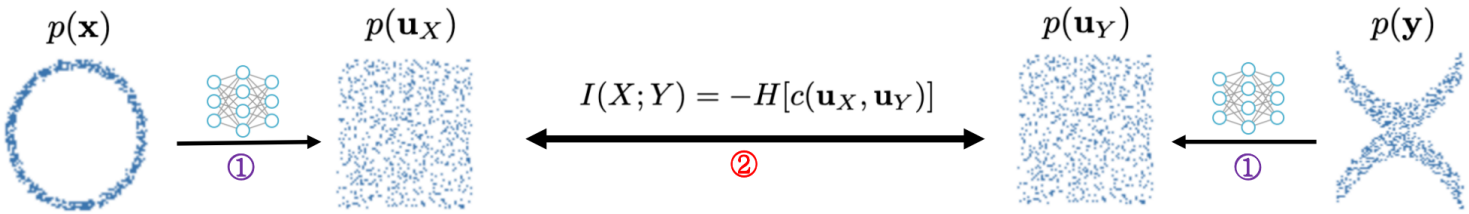
- Two steps

Divide and conquer approach



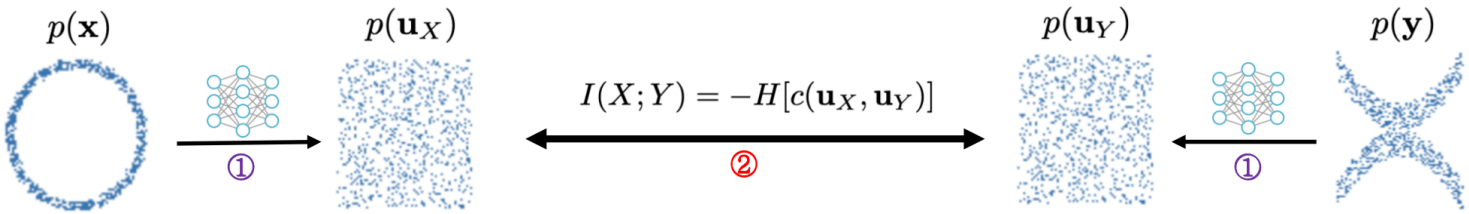
- Two steps
  1. Learn, separately, mappings $\hat{G}_x$ and $\hat{G}_y$ such that $\hat{\mathbf{u}}_x = \hat{G}_x(\mathbf{x})$ and $\hat{\mathbf{u}}_y = \hat{G}_y(\mathbf{y})$ and are uniformly distributed on the unit cube.

Divide and conquer approach



- Two steps
  1. Learn, separately, mappings $\hat{G}_x$ and $\hat{G}_y$ such that $\hat{\mathbf{u}}_x = \hat{G}_x(\mathbf{x})$ and $\hat{\mathbf{u}}_y = \hat{G}_y(\mathbf{y})$ and are uniformly distributed on the unit cube.
  2. Estimate their joint distribution $\hat{c}(\hat{\mathbf{u}}_x, \hat{\mathbf{u}}_y)$

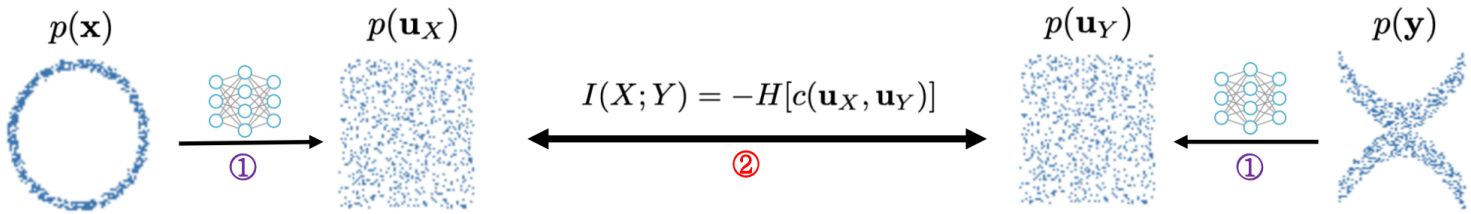
Divide and conquer approach



- Two steps
  1. Learn, separately, mappings $\hat{G}_x$ and $\hat{G}_y$ such that $\hat{\mathbf{u}}_x = \hat{G}_x(\mathbf{x})$ and $\hat{\mathbf{u}}_y = \hat{G}_y(\mathbf{y})$ and are uniformly distributed on the unit cube.
  2. Estimate their joint distribution $\hat{c}(\hat{\mathbf{u}}_x, \hat{\mathbf{u}}_y)$
- An estimate of the MI is given by the log-likelihood of $\hat{c}$, e.g. on hold-out data.

$$\hat{I} = \frac{1}{n'} \sum_{i=1}^{n'} \log \hat{c}(\hat{\mathbf{u}}_x^{(i)}, \hat{\mathbf{u}}_y^{(i)}) \tag{17}$$

Divide and conquer approach



- Two steps
  1. Learn, separately, mappings $\hat{G}_x$ and $\hat{G}_y$ such that $\hat{\mathbf{u}}_x = \hat{G}_x(\mathbf{x})$ and $\hat{\mathbf{u}}_y = \hat{G}_y(\mathbf{y})$ and are uniformly distributed on the unit cube.
  2. Estimate their joint distribution $\hat{c}(\hat{\mathbf{u}}_x, \hat{\mathbf{u}}_y)$
- An estimate of the MI is given by the log-likelihood of $\hat{c}$, e.g. on hold-out data.

$$\hat{I} = \frac{1}{n'} \sum_{i=1}^{n'} \log \hat{c}(\hat{\mathbf{u}}_x^{(i)}, \hat{\mathbf{u}}_y^{(i)}) \tag{17}$$

- Note: the first step does not require joint data.

Complexity control

- Approach allows us to use models of different complexity for the marginals (learning the mappings G_x and G_y) and the dependencies (learning c).

Complexity control

- Approach allows us to use models of different complexity for the marginals (learning the mappings G_x and G_y) and the dependencies (learning c).
- To learn G_x and G_y, we use flexible normalising flows.

Complexity control

- Approach allows us to use models of different complexity for the marginals (learning the mappings G_x and G_y) and the dependencies (learning c).
- To learn G_x and G_y, we use flexible normalising flows.
- The data space for learning c is higher dimensional, $d_x + d_y$, rather than d_x or d_y when learning G_x or G_y.

Complexity control

- Approach allows us to use models of different complexity for the marginals (learning the mappings G_x and G_y) and the dependencies (learning c).
- To learn G_x and G_y, we use flexible normalising flows.
- The data space for learning c is higher dimensional, $d_x + d_y$, rather than d_x or d_y when learning G_x or G_y.
- We control the complexity when learning c using model selection.

Complexity control

- Approach allows us to use models of different complexity for the marginals (learning the mappings G_x and G_y) and the dependencies (learning c).
- To learn G_x and G_y, we use flexible normalising flows.
- The data space for learning c is higher dimensional, $d_x + d_y$, rather than d_x or d_y when learning G_x or G_y.
- We control the complexity when learning c using model selection.
- Here: we use Gaussian mixture models and determine the number of mixtures using validation data.

Side-note: Normalizing flows

- Basic idea: learn an invertible transformation that maps a complex data distribution, e.g. $p(\mathbf{x})$, to a simple one (here: Gaussian, then mapped to uniform).

Side-note: Normalizing flows

- Basic idea: learn an invertible transformation that maps a complex data distribution, e.g. $p(\mathbf{x})$, to a simple one (here: Gaussian, then mapped to uniform).
- Transformation is often a sequence of parameterised transformations with easy-to-compute Jacobians.

Side-note: Normalizing flows

- Basic idea: learn an invertible transformation that maps a complex data distribution, e.g. $p(\mathbf{x})$, to a simple one (here: Gaussian, then mapped to uniform).
- Transformation is often a sequence of parameterised transformations with easy-to-compute Jacobians.
- More expressive version: use continuous-time flow.

Side-note: Normalizing flows

- Basic idea: learn an invertible transformation that maps a complex data distribution, e.g. $p(\mathbf{x})$, to a simple one (here: Gaussian, then mapped to uniform).
- Transformation is often a sequence of parameterised transformations with easy-to-compute Jacobians.
- More expressive version: use continuous-time flow.
- Define dynamics with an ODE:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}_\theta(\mathbf{x}_t, t), \qquad \mathbf{x}_0 \sim \mathcal{N}(0, 1) \tag{18}$$

Side-note: Normalizing flows

- Basic idea: learn an invertible transformation that maps a complex data distribution, e.g. $p(\mathbf{x})$, to a simple one (here: Gaussian, then mapped to uniform).
- Transformation is often a sequence of parameterised transformations with easy-to-compute Jacobians.
- More expressive version: use continuous-time flow.
- Define dynamics with an ODE:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}_\theta(\mathbf{x}_t, t), \qquad \mathbf{x}_0 \sim \mathcal{N}(0, 1) \tag{18}$$

- Log-density evolution follows (continuity equation):

$$\frac{d}{dt} \log p_t(\mathbf{x}_t) = -\nabla \cdot \mathbf{v}_\theta(\mathbf{x}_t, t) \tag{19}$$

Side-note: Normalizing flows

- Basic idea: learn an invertible transformation that maps a complex data distribution, e.g. $p(\mathbf{x})$, to a simple one (here: Gaussian, then mapped to uniform).
- Transformation is often a sequence of parameterised transformations with easy-to-compute Jacobians.
- More expressive version: use continuous-time flow.
- Define dynamics with an ODE:

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}_\theta(\mathbf{x}_t, t), \qquad \mathbf{x}_0 \sim \mathcal{N}(0, 1) \tag{18}$$

- Log-density evolution follows (continuity equation):

$$\frac{d}{dt} \log p_t(\mathbf{x}_t) = -\nabla \cdot \mathbf{v}_\theta(\mathbf{x}_t, t) \tag{19}$$

- Parameter $\boldsymbol{\theta}$ can be learned by maximum likelihood or flow-matching (Lipman et al, 2022).

Reinterpreting existing approaches

- Density estimation: When learning $p(\mathbf{x}, \mathbf{y})$ with a single model, the model needs to simultaneously account for both

Reinterpreting existing approaches

- Density estimation: When learning $p(\mathbf{x}, \mathbf{y})$ with a single model, the model needs to simultaneously account for both
  - the marginal properties $p(\mathbf{x})$ and $p(\mathbf{y})$

Reinterpreting existing approaches

- Density estimation: When learning $p(\mathbf{x}, \mathbf{y})$ with a single model, the model needs to simultaneously account for both
  - the marginal properties $p(\mathbf{x})$ and $p(\mathbf{y})$
  - the dependencies $c(\mathbf{u}_x, \mathbf{u}_y)$

Reinterpreting existing approaches

- Density estimation: When learning $p(\mathbf{x}, \mathbf{y})$ with a single model, the model needs to simultaneously account for both
  - the marginal properties $p(\mathbf{x})$ and $p(\mathbf{y})$
  - the dependencies $c(\mathbf{u}_x, \mathbf{u}_y)$
- Our approach explicitly separates the two, giving more control.

Reinterpreting existing approaches

- Density estimation: When learning $p(\mathbf{x}, \mathbf{y})$ with a single model, the model needs to simultaneously account for both
  - the marginal properties $p(\mathbf{x})$ and $p(\mathbf{y})$
  - the dependencies $c(\mathbf{u}_x, \mathbf{u}_y)$
- Our approach explicitly separates the two, giving more control.
- Ratio estimation: The methods approximate the pointwise MI $p(\mathbf{x}, \mathbf{y})/(p(\mathbf{x})p(\mathbf{y}))$ by contrastive learning or MI-lower bound maximisation in the $(\mathbf{x}, \mathbf{y})$ space, normally without complexity control.

Reinterpreting existing approaches

- Density estimation: When learning $p(\mathbf{x}, \mathbf{y})$ with a single model, the model needs to simultaneously account for both
  - the marginal properties $p(\mathbf{x})$ and $p(\mathbf{y})$
  - the dependencies $c(\mathbf{u}_x, \mathbf{u}_y)$
- Our approach explicitly separates the two, giving more control.
- Ratio estimation: The methods approximate the pointwise MI $p(\mathbf{x}, \mathbf{y})/(p(\mathbf{x})p(\mathbf{y}))$ by contrastive learning or MI-lower bound maximisation in the $(\mathbf{x}, \mathbf{y})$ space, normally without complexity control.
- Our approach treats the ratio as a density in the standardised $(\mathbf{u}_x, \mathbf{u}_y)$ space, enabling standard model selection and learning by MLE, which provides the most efficient estimators.

Program

Motivation

Background

New MI estimation framework

Simulation results

Main comparison methods

- Density estimation:

Main comparison methods

- Density estimation:
  - MI Neural Diffusion Estimation (MINDE, Franzese et al, 2023): uses diffusion models

Main comparison methods

- Density estimation:
  - MI Neural Diffusion Estimation (MINDE, Franzese et al, 2023): uses diffusion models
  - MI Estimation via Normalising Flows ($\mathcal{N}$-MIENF, Butakov et al, 2024): Connects to our approach if only one Gaussian mixture component is used, learns all models jointly and treats the pointwise MI as ratio.

Main comparison methods

- Density estimation:
  - MI Neural Diffusion Estimation (MINDE, Franzese et al, 2023): uses diffusion models
  - MI Estimation via Normalising Flows ($\mathcal{N}$-MIENF, Butakov et al, 2024): Connects to our approach if only one Gaussian mixture component is used, learns all models jointly and treats the pointwise MI as ratio.
- Ratio estimation:

Main comparison methods

- Density estimation:
  - MI Neural Diffusion Estimation (MINDE, Franzese et al, 2023): uses diffusion models
  - MI Estimation via Normalising Flows ($\mathcal{N}$-MIENF, Butakov et al, 2024): Connects to our approach if only one Gaussian mixture component is used, learns all models jointly and treats the pointwise MI as ratio.
- Ratio estimation:
  - MI Neural Estimation (MINE, Belghazi et al, 2018): maximises a variational lower bounds on the MI.

Main comparison methods

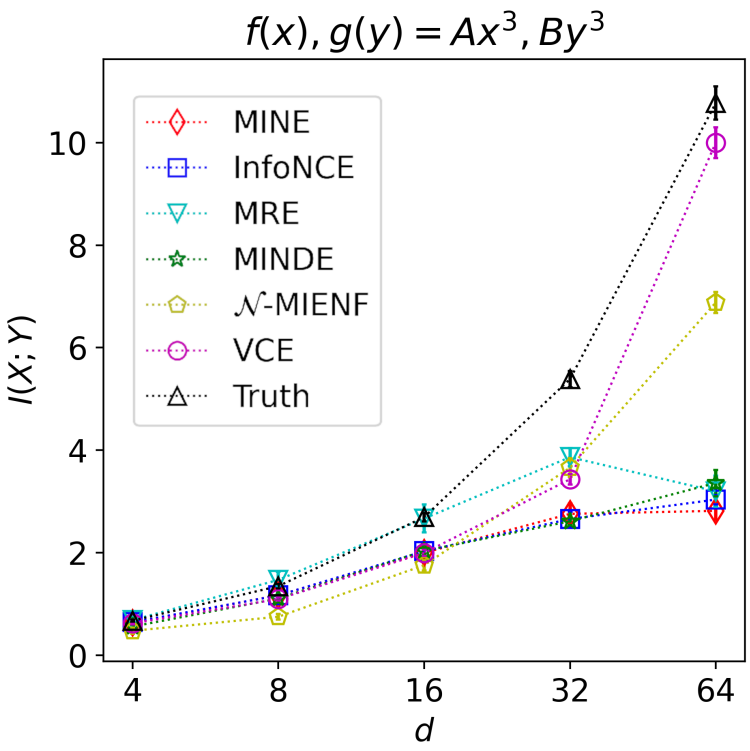
- Density estimation:
  - MI Neural Diffusion Estimation (MINDE, Franzese et al, 2023): uses diffusion models
  - MI Estimation via Normalising Flows ($\mathcal{N}$-MIENF, Butakov et al, 2024): Connects to our approach if only one Gaussian mixture component is used, learns all models jointly and treats the pointwise MI as ratio.
- Ratio estimation:
  - MI Neural Estimation (MINE, Belghazi et al, 2018): maximises a variational lower bounds on the MI.
  - Information NCE (InfoNCE, van den Oord, 2018): frames MI estimation as a contrastive learning problem, provides a lower bound on the MI.

Main comparison methods

- Density estimation:
  - MI Neural Diffusion Estimation (MINDE, Franzese et al, 2023): uses diffusion models
  - MI Estimation via Normalising Flows ($\mathcal{N}$-MIENF, Butakov et al, 2024): Connects to our approach if only one Gaussian mixture component is used, learns all models jointly and treats the pointwise MI as ratio.
- Ratio estimation:
  - MI Neural Estimation (MINE, Belghazi et al, 2018): maximises a variational lower bounds on the MI.
  - Information NCE (InfoNCE, van den Oord, 2018): frames MI estimation as a contrastive learning problem, provides a lower bound on the MI.
  - Multinomial ratio estimation (MRE, Srivastava et al 2023): improved version of telescopic density ratio estimation (TRE, Rhodes et al, 2020) that uses auxiliary distributions to bridge the gap between $p(\mathbf{x}, \mathbf{y})$ and $p(\mathbf{x})p(\mathbf{y})$.

Synthetic data (More results in the paper!)

- Left: Correlated Gaussian random variables x, y ($\rho = 0.7$) are nonlinearly transformed and mixed.
- Right: Student-t random variables x, y ($\rho = 0.7$) with 1 degree of freedom.
- Proposed method: Vector copula-based MI estimator (VCE)



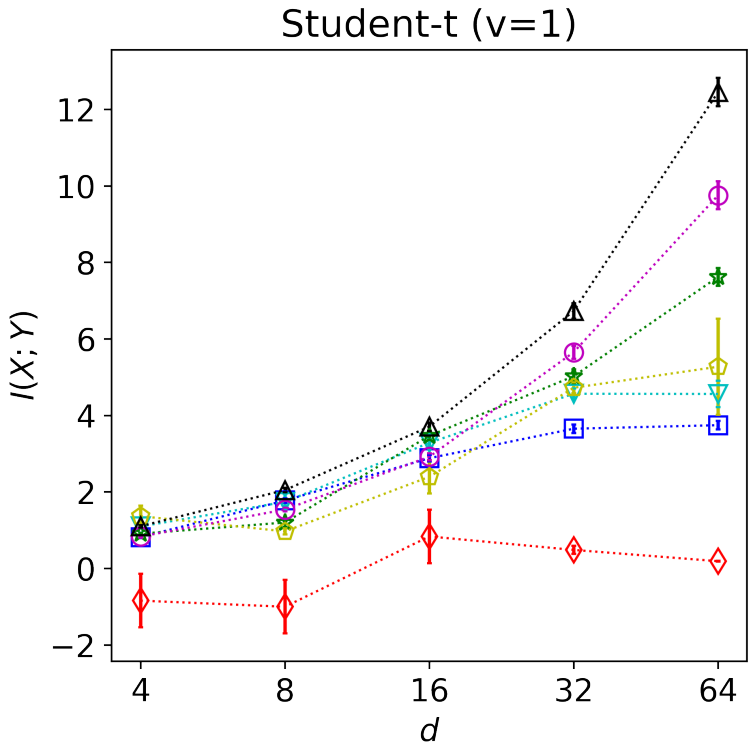
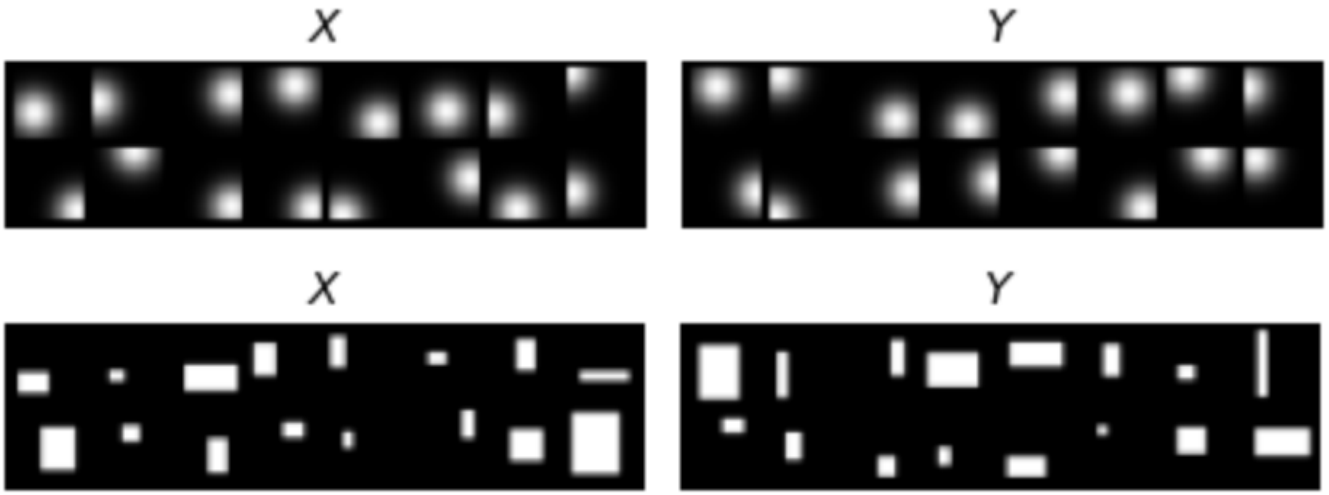


Image data (Butakov et al, 2024)

- Generated images based on random variables with known MI.
- If mapping from random variables to images is injective and piece-wise smooth, MI stays the same.
- Top: the center coordinate of the blob has been derived from a Gaussian.
- Bottom: the coordinates of the rectangles have been derived from a Gaussian.



Results on the image data

- Proposed VCE is best for blobs, still reasonable for rectangles.
- Comparison to $\mathcal{N}$-MIENF highlights the importance of using a non-Gaussian copula model (mixture model in our case)
- MRE performs well for rectangles but may overshoot (1.23 vs 1 in case of blob data)

Method	**Blobs**			**Rectangles**		
	$I(X;Y)=1$	$I(X;Y)=3$	$I(X;Y)=7$	$I(X;Y)=1$	$I(X;Y)=3$	$I(X;Y)=7$
MINE	0.89 ± 0.07	2.86 ± 0.24	5.46 ± 0.27	0.81 ± 0.13	$\mathbf{2.57 \pm 0.26}$	5.39 ± 0.23
InfoNCE	0.86 ± 0.14	2.63 ± 0.13	3.83 ± 0.12	0.78 ± 0.17	2.49 ± 0.28	3.86 ± 0.15
MRE	1.23 ± 0.16	2.85 ± 0.21	5.91 ± 0.28	0.82 ± 0.24	2.56 ± 0.48	$\mathbf{5.45 \pm 0.31}$
$\mathcal{N}$-MIENF	0.74 ± 0.12	2.42 ± 0.16	3.85 ± 0.22	0.54 ± 0.13	0.76 ± 0.14	1.54 ± 0.11
VCE	$\mathbf{0.92 \pm 0.04}$	$\mathbf{2.93 \pm 0.12}$	$\mathbf{6.53 \pm 0.36}$	$\mathbf{0.83 \pm 0.12}$	2.27 ± 0.23	5.02 ± 0.14

Results on text embeddings

- The data are pairs of embeddings computed on the Internet Movie Database (IMDB, Maas et al, 2011)
- Ground truth MI is unknown but can be estimated using the labels. MI relatively low, making task easier.
- Similar points as for images: VCE performs better than $\mathcal{N}$-MIENF, MRE may overshoot.

	X	Y
1	(positive) I thought this was a wonderful way to spend time on ...	(positive) If you like original gut wrenching laughter you will like ...
2	(negative) So im not a big fan of Boll's work but then ...	(positive) This a fantastic movie of three prisoners who become famous...

Method	$I(X;Y) \approx 2.1$	$I(X;Y) \approx 0.9$
MINE	1.83 ± 0.04	0.71 ± 0.05
InfoNCE	1.64 ± 0.09	0.70 ± 0.06
MRE	1.72 ± 0.07	1.23 ± 0.02
$\mathcal{N}$-MIENF	0.91 ± 0.05	0.43 ± 0.03
VCE	$\mathbf{2.01 \pm 0.04}$	$\mathbf{0.83 \pm 0.01}$

(a) Llama-3 13B

Method	$I(X;Y) \approx 1.5$	$I(X;Y) \approx 0.2$
MINE	$\mathbf{1.42 \pm 0.04}$	0.18 ± 0.02
InfoNCE	1.41 ± 0.03	0.19 ± 0.04
MRE	1.23 ± 0.09	0.31 ± 0.09
$\mathcal{N}$-MIENF	0.73 ± 0.03	0.11 ± 0.02
VCE	1.22 ± 0.02	$\mathbf{0.19 \pm 0.02}$

(b) BERT

Conclusions

- Estimating the MI is a fundamental and widely occurring task.
- There are two broad approaches to MI estimation:
  - density estimation
  - ratio estimation
- We here proposed a third approach based on vector copulas.
- Practical benefits:
  - Disentangles marginal from dependence structure
  - Dependence structure can be estimated via MLE
  - Flexible models for marginals, complexity control for dependence structure
- New perspective: Pointwise MI ratio can be seen as a density; MI is its (negative) entropy.
- Not always best, but provides robust and accurate MI estimates on the tested data sets.